

Cryptography Lab 02 & 03: Block Cipher Operative Modes & Feistel Network Design

Authors: AJANA Chadi & LAZREK Mohammed Amine

Focus: ECB vs. CBC Operational Security, Error Propagation Mechanics, Feistel Architectures, and Python Implementations.

1. Block Cipher Operational Modes (ECB vs. CBC)

Electronic Codebook (ECB) Mode Analysis

- **Operational Mechanics:** ECB mode encrypts each block of data independently using the identical key configuration.
- **Security Limitation:** Because it lacks randomization, identical plaintext blocks always yield identical ciphertext blocks. This preserves structural patterns in the underlying data, making it vulnerable to pattern analysis attacks (such as revealing structures in encrypted image data).

Plaintext Block 1 ==> [Key Encryption] ==> Ciphertext Block 1

Plaintext Block 2 ==> [Key Encryption] ==> Ciphertext Block 2 (Identical block patterns remain visible)

Cipher Block Chaining (CBC) Mode Analysis

- **Operational Mechanics:** CBC mode introduces feedback by XORing each plaintext block with the preceding ciphertext block prior to encryption. This ensures that identical plaintext blocks produce entirely different ciphertext blocks throughout the sequence.
- **Error Propagation Properties:** If a single transmission bit error corrupts the first block

of ciphertext (C_1), the error propagates during decryption:

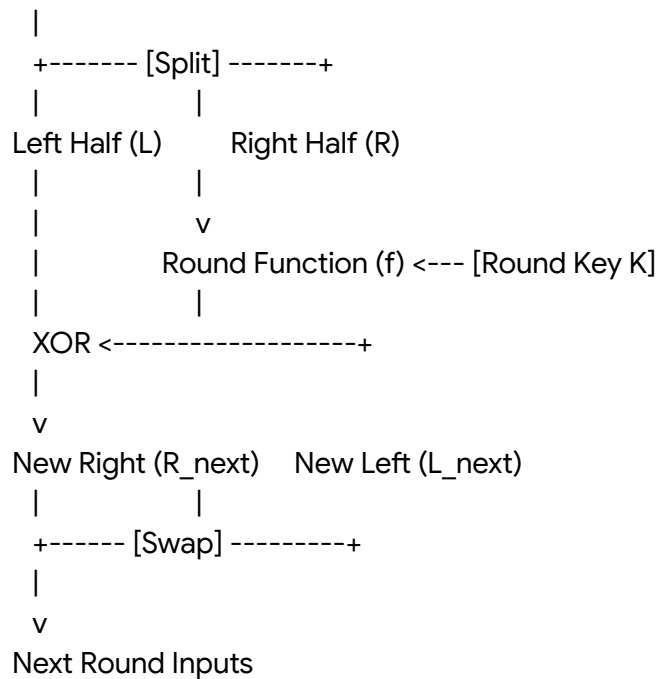
- **Plaintext Block 1 (P_1):** Completely corrupted and unreadable due to the direct error in C_1 .
- **Plaintext Block 2 (P_2):** Corrupted exactly at the specific bit position where the error occurred in C_1 , because C_1 is XORed directly into the output of the decrypted C_2 block.

- **Subsequent Blocks** ($P_3, P_4, \dots$): Unaffected and completely intact, because the decryption of C_3 relies on C_2 , which suffered no transmission errors.
- **Primary Use Case:** CBC mode is ideal for encrypting sequential data streams or long message payloads where pattern obfuscation is required.

2. Structural Architecture of a Feistel Cipher Layout

A Feistel cipher is an iterative cryptographic structure that divides a data block into two equal halves, processing them through multiple rounds of substitution and permutation. A key advantage of this design is that encryption and decryption routines share an identical structural architecture; reversing the execution order of the subkeys is all that is required to decrypt the ciphertext.

Plaintext Block



Algorithmic Execution Steps (Per Round):

1. **Split:** The 8-bit ASCII binary string input is divided into two 4-bit segments: Left (L_i) and Right (R_i).
2. **Function Processing:** The right sub-block (R_i) is processed through a round function f alongside a round subkey (K_i).

3. **XOR Operation:** The output of function f is combined with the left sub-block (L_i) via an XOR operation:

$$R_{i+1} = L_i \oplus f(R_i, K_i)$$

4. **Feistel Swap:** The left sub-block for the next round inherits the current right sub-block:

$$L_{i+1} = R_i$$

5. **Iteration:** This sequence repeats across all designated rounds, scrambling the data structure effectively.

3. Python Jupyter Notebook Implementation of a 2-Round Feistel Cipher

Below is the complete, runnable Python source code implementing the 2-round Feistel network execution structure.

```
import random
```

```
def text_to_binary(text):
```

```
    """Converts a standard text string into an 8-bit binary per-character representation."""
    return ''.join(format(ord(char), '08b') for char in text)
```

```
def binary_to_text(binary_str):
```

```
    """Converts an 8-bit binary block string back into standard text characters."""
    chars = []
    for i in range(0, len(binary_str), 8):
        byte = binary_str[i:i+8]
        if len(byte) == 8:
            chars.append(chr(int(byte, 2)))
    return ''.join(chars)
```

```
def xor_operation(bits1, bits2):
```

```
    """Performs a bitwise XOR operation between two binary strings of equal length."""
    return ''.join(str(int(b1) ^ int(b2)) for b1, b2 in zip(bits1, bits2))
```

```
def generate_round_keys(half_length):
```

```
    """Generates random binary keys matched to the size of the sub-block."""
    key1 = ''.join(random.choice(['0', '1']) for _ in range(half_length))
    key2 = ''.join(random.choice(['0', '1']) for _ in range(half_length))
    return key1, key2
```

```
def feistel_round(left, right, round_key):
```

```

"""Executes a single structural round of the Feistel network."""
# Round Function:  $f(R, K) = R \text{ XOR } K$ 
f_output = xor_operation(right, round_key)
# New Right Half:  $L \text{ XOR } f(R, K)$ 
new_right = xor_operation(left, f_output)
# New Left Half: Inherits old Right Half
new_left = right
return new_left, new_right

def encrypt_feistel(plaintext_block, key1, key2):
    """Runs encryption across a 2-round Feistel infrastructure pipeline."""
    midpoint = len(plaintext_block) // 2
    L1 = plaintext_block[:midpoint]
    R1 = plaintext_block[midpoint:]

    # Round 1 Execution
    L2, R2 = feistel_round(L1, R1, key1)

    # Round 2 Execution
    L3, R3 = feistel_round(L2, R2, key2)

    # Final Block Construction (Combine without swapping after the final round)
    return L3 + R3

def decrypt_feistel(ciphertext_block, key1, key2):
    """Decrypts by executing the rounds with the keys in reverse order (Key2 then Key1)."""
    midpoint = len(ciphertext_block) // 2
    L3 = ciphertext_block[:midpoint]
    R3 = ciphertext_block[midpoint:]

    # Decryption Round 1 uses Key 2
    L2, R2 = feistel_round(L3, R3, key2)

    # Decryption Round 2 uses Key 1
    L1, R1 = feistel_round(L2, R2, key1)

    return L1 + R1

# Execution Demonstration Verification
if __name__ == "__main__":
    sample_text = "Hi"
    print(f"Original Input Plaintext Target: '{sample_text}'")

```

```
# Transformation to Binary Space
binary_payload = text_to_binary(sample_text)
print(f"Binary Structural Form (16 bits total): {binary_payload}")

# Subkey Generation Routine
block_half_size = len(binary_payload) // 2
k1, k2 = generate_round_keys(block_half_size)
print(f"Generated Key Assets: K1={k1}, K2={k2}")

# Encryption Execution
cipher_bits = encrypt_feistel(binary_payload, k1, k2)
print(f"Output Encrypted Ciphertext Bits: {cipher_bits}")

# Decryption Verification Pipeline
decrypted_bits = decrypt_feistel(cipher_bits, k1, k2)
recovered_text = binary_to_text(decrypted_bits)
print(f"Successfully Recovered Plaintext Text: '{recovered_text}'")
```